从 Q-learning 到 Soft Actor-Critic：完整推导与设计动机

课程首页 · English · PDF

项目	说明
适用范围	Q-learning、TD/SARSA、DQN、Soft Bellman 方程、现代 SAC、双 Critic、重参数化与自动熵调节；教学伪代码
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

核验边界： 仅进行文档整合与排版检查；未进行训练或真机验证。不包含实验章节。 固定源码引用见资料清单。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

目录

- 导读：这份教程要回答什么？
- 1. 从 MDP 到 V 与 Q：为什么需要价值函数？
  - 1.1 强化学习目标
  - 1.2 状态价值与动作价值
  - 1.3 Bellman Expectation Equation：把长期问题拆成一步
  - 1.4 Policy Evaluation 与 Policy Improvement
- 2. 从 Bellman Expectation 到 Bellman Optimality
  - 2.1 最优价值与最优性方程
  - 2.2 Bellman 算子为何重要？
  - 2.3 一个两动作例子
- 3. TD Learning、SARSA 与表格 Q-learning
  - 3.1 蒙特卡罗与 TD：为什么不必等轨迹结束？
  - 3.2 SARSA：评估执行中的策略
  - 3.3 Q-learning：行为可以探索，目标采用贪心
  - 3.4 三种 Target 并排比较
  - 3.5 何时能保证收敛？
- 4. DQN：把表格 Q-learning 扩展到大状态空间
  - 4.1 从表格到神经网络的 TD Regression
  - 4.2 Replay Buffer 为什么有用？
  - 4.3 Target Network 为什么有用？
  - 4.4 为什么最大化会放大 Q 高估？
  - 4.5 Double Q-learning、Double DQN 与 SAC 的双 Q 不是同一公式
  - 4.6 为什么深度价值学习仍可能不稳定？

- 5. 连续动作为什么推动我们走向 Actor-Critic?
  - 5.1 动作最大化成为内层优化问题
  - 5.2 Actor 是一个可学习的动作优化器
  - 5.3 为什么 SAC 进一步使用随机 Actor?
- 6. 最大熵强化学习：从奖励最大化到奖励与熵联合优化
  - 6.1 策略熵与温度
  - 6.2 SAC 采用的 Soft Q 与 Soft V 约定
- 7. Hard Bellman 与 Soft Bellman：从 Max 到 LogSumExp
  - 7.1 有限离散动作中的最优 Soft Value
  - 7.2 为什么 $\alpha \to 0$ 时恢复 Max?
  - 7.3 连续动作时的积分与边界条件
  - 7.4 Soft Optimality 与 Soft Policy Evaluation 必须分开
- 8. Soft Policy Improvement：为什么出现 Boltzmann Policy 与 KL?
  - 8.1 从最优分布到可训练的策略
  - 8.2 反向 KL 将不可计算的归一化常数消去
  - 8.3 精确 Soft Policy Iteration 的改进性质
- 9. 从 Soft Policy Iteration 到现代 SAC
  - 9.1 两个循环对应两个神经网络
  - 9.2 为什么 SAC 可以复用历史数据?
  - 9.3 与 PPO 的 On-Policy 属性区分
- 10. 完整 SAC Critic：Target Network、双 Q 与 TD 回归
  - 10.1 从 Soft Bellman 到单样本 TD Target
  - 10.2 双 Q 的取最小值解决什么?
  - 10.3 时间截断与真正终止
  - 10.4 目标网络的软更新
- 11. 完整 SAC Actor：重参数化、动作边界与概率修正
  - 11.1 从 KL 投影到 Actor Loss
  - 11.2 Reparameterization Trick
  - 11.3 Tanh Gaussian 的 Log Probability 必须修正
  - 11.4 从归一化动作到机器人实际动作
- 12. Automatic Entropy Tuning：为什么学习温度 α?
  - 12.1 目标熵约束与拉格朗日乘子
  - 12.2 温度损失与方向检查
  - 12.3 为什么优化 $\eta = \log \alpha$?
- 13. 现代 SAC 的完整算法流程
  - 13.1 主要模块和三个损失
  - 13.2 一次更新究竟发生了什么?
  - 13.3 SAC v1 与现代 SAC：是否显式学习 V?
- 14. 算法之间到底有什么联系?
  - 14.1 Q-learning 与 SAC 的最关键区别

    - 14.2 PPO 与 SAC：Score Function 和 Pathwise Gradient
- 15. 机器人控制中的适用边界与常见误解
    - 15.1 为什么机器人任务特别需要重视 Q 外推?
    - 15.2 关节位置、速度、力矩与归一化动作
    - 15.3 什么情况下不宜直接套用标准 SAC?
    - 15.4 五个常见误解
- 16. 全部推导的最短闭环
- 参考资料与延伸阅读

> **阅读定位**。 本文以原有《SAC（Soft Actor-Critic）完整推导与设计动机》为基础重新组织，从经典 Q-learning 出发，逐步推导现代连续动作 SAC。重视“问题 → 数学表达 → 算法设计”的因果关系，不包含实验章节。本文使用统一的 Q/V 与终止标记约定，避免普通 Bellman、Soft Bellman 和最优性方程之间的概念混淆。
>
> **原课程：** 课程首页 · 原中文教程 · 原英文教程。本修订版已收录于课程仓库；原始版本引用固定到修订前的 commit，便于溯源。

导读：这份教程要回答什么?

- 为什么 $Q(s,a)$ 能指导控制，以及 Q 与 V 有何不同?
- 为什么 Q-learning 不需要先知道环境动力学，也可以使用其他策略产生的数据?
- DQN 为什么需要 Replay Buffer、Target Network，Double Q 又在修复什么?
- 为什么连续机器人动作不能简单地照搬 $\max_a Q(s,a)$?
- 传统 Bellman Optimality 如何变成 Soft Bellman Optimality，再如何通向 SAC 的 Critic 与 Actor?
- 自动熵调节、重参数化和双 Critic 各自解决什么问题?

知识路线： Bellman Expectation → Bellman Optimality → TD / Q-learning → DQN → 连续动作 Actor-Critic → Maximum-Entropy RL → Soft Policy Iteration → SAC。

统一记号。 s 为状态，a 为动作，$r(s,a)$ 为期望一步奖励，$P(s' \mid s,a)$ 为环境转移核，$\pi(a \mid s)$ 为策略，$\gamma \in [0,1)$ 为折扣，$d=1$ 表示真实终止（非单纯时间截断）。Q^π, V^π 表示给定策略的价值，Q^*, V^* 表示最优价值；加入熵时特别加“soft”描述，尽管部分公式为简洁省略上标。

---

1. 从 MDP 到 V 与 Q：为什么需要价值函数?

1.1 强化学习目标

折扣马尔可夫决策过程写为

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma), \qquad 0 \le \gamma < 1.$$

一条轨迹 $\tau = (s_0, a_0, r_0, s_1, \dots)$ 的折扣回报为

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \qquad J(\pi) = \mathbb{E}_{\tau\sim\pi}[G_0].$$

注意 $J(\pi)$ 还隐含初始状态分布；实际机器人任务中，初始物体位置、姿态或机器人关节状态常有随机性。**环境奖励** r 和 **价值估计** Q 不是一回事：奖励告诉我们这一步怎样，价值估计还考虑以后的影响。

1.2 状态价值与动作价值

给定策略 π：

$$\boxed{V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \qquad Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a].}$$

直观地，$V^\pi(s)$ 问“按照 π 继续，当前状态值多少”；$Q^\pi(s,a)$ 问“现在先执行动作 a，之后按照 π，总共值多少”。后者允许直接比较不同动作。

由全期望公式，离散动作空间中

$$\boxed{V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s,a).}$$

连续动作时，把求和换成 $\int_{\mathcal{A}} \pi(a \mid s) Q^\pi(s,a)\, da$。一个抓取动作当前奖励可能是零，但如果它显著提高后续拿起物体的成功率，其 Q 仍然可以很高。

1.3 Bellman Expectation Equation：把长期问题拆成一步

利用 $G_t = r_t + \gamma G_{t+1}$ 和马尔可夫性质：

$$\boxed{Q^\pi(s,a) = r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot\mid s,a)}[V^\pi(s')].}$$

再代入 $V^\pi(s') = \mathbb{E}_{a'\sim\pi}[Q^\pi(s',a')]$：

$$\boxed{Q^\pi(s,a) = r(s,a) + \gamma \mathbb{E}_{s',a'\sim P,\pi}[Q^\pi(s',a')].}$$

这叫 **Bellman Expectation Equation**：评估的是一个**给定**策略，并未在动作空间求最大值。V^π 也有对应关系：$V^\pi(s) = \mathbb{E}_{a\sim\pi, s'\sim P}[r(s,a) + \gamma V^\pi(s')]$。

1.4 Policy Evaluation 与 Policy Improvement

两步组成经典 Policy Iteration：**评估** $\pi_k \mapsto Q^{\pi_k}$；**改进** $\pi_{k+1}(s) \in \arg\max_a Q^{\pi_k}(s,a)$。评估问当前策略有多好；改进问如何利用当前 Q 选更好的动作。后面的 SAC 保留这两步结构，但将贪心动作改成熵正则化的随机策略。

2. 从 Bellman Expectation 到 Bellman Optimality

2.1 最优价值与最优性方程

定义 $V^*(s) = \sup_\pi V^\pi(s)$，$Q^*(s,a) = \sup_\pi Q^\pi(s,a)$。对有限离散动作集合（最大值存在时），有

$$\boxed{V^*(s) = \max_a Q^*(s,a),}$$

$$\boxed{Q^*(s,a) = r(s,a) + \gamma \mathbb{E}_{s'\sim P}[\max_{a'} Q^*(s',a')].}$$

这是 **Bellman Optimality Equation**。与上节最明显的区别，是下一步使用 $\max_{a'}$ 而不是 $\mathbb{E}_{a'\sim\pi}$。它刻画最优价值函数的固定点，而不再需要给定一个未来策略。

2.2 Bellman 算子为何重要?

定义最优 Bellman 算子

$$(\mathcal{T}^*Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}\left[\max_{a'} Q(s',a')\right].$$

在有界奖励、折扣 MDP 与有界 Q 等常见条件下，利用 $|\max_i x_i - \max_i y_i| \le \max_i |x_i - y_i|$，可得

$$\boxed{\|\mathcal{T}^*Q_1 - \mathcal{T}^*Q_2\|_\infty \le \gamma \|Q_1 - Q_2\|_\infty.}$$

因为 $\gamma < 1$，这个压缩性质保证唯一固定点 Q^*；精确同步 Value Iteration $Q_{k+1} = \mathcal{T}^*Q_k$ 会收敛。**注意：**这是对精确算子与适当函数空间的结论，不代表一般深度 Q 网络训练必然收敛。

2.3 一个两动作例子

某状态有两种抓取方式，假设它们立即得到的奖励均为 0；左抓通往的下一状态最优价值为 8，右抓为 10，$\gamma = 0.9$。则

$$Q^*(s,a_L) = 0 + 0.9 \times 8 = 7.2, \qquad Q^*(s,a_R) = 0 + 0.9 \times 10 = 9.$$

虽然这一步没有奖励，Q 仍然倾向右抓。这解释了为何机器人长时程任务不能仅根据即时奖励选择动作。

3. TD Learning、SARSA 与表格 Q-learning

3.1 蒙特卡罗与 TD：为什么不必等轨迹结束?

蒙特卡罗（Monte Carlo, MC）通过完整回报 G_t 估计价值；Temporal-Difference（TD）则利用“一步奖励 + 下一状态估计”提前更新。例如策略评估的 TD(0)：

$$\delta_t^V = r_t + \gamma(1-d_t)V(s_{t+1}) - V(s_t),$$

$$V(s_t) \leftarrow V(s_t) + \eta_t \delta_t^V.$$

TD 的优势是**不必等待回合结束**；代价是 Target 中包含已有估计，因此会引入自举（Bootstrapping）偏差和误差传播。TD 与 MC 不是严格的“有偏/无偏”简单二分，误差还取决于采样、函数逼近及估计对象。

3.2 SARSA：评估执行中的策略

SARSA 用实际执行策略抽样得到的下一动作 a_{t+1}：

$$\boxed{y_t^{\text{SARSA}} = r_t + \gamma(1-d_t)Q(s_{t+1},a_{t+1}).}$$

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \eta_t\big(y_t^{\text{SARSA}} - Q(s_t,a_t)\big).$$

当 a_{t+1} 由正在执行的 ε-greedy 策略产生时，标准 SARSA 是 **On-Policy TD Control**：目标中的下一动作也反映了该探索策略。

3.3 Q-learning：行为可以探索，目标采用贪心

Watkins 的表格 Q-learning 使用

$$\boxed{y_t^Q = r_t + \gamma(1-d_t)\max_{a'} Q_t(s_{t+1}, a'),}$$

$$\boxed{Q_{t+1}(s_t,a_t) = Q_t(s_t,a_t) + \eta_t\left[y_t^Q - Q_t(s_t,a_t)\right].}$$

将方括号记为 TD Error δ_t，更新就是“旧估计 + 学习率 × 预测误差”。执行动作可由 ε-greedy 策略产生：以概率 $1-\varepsilon$ 选择贪心动作，以概率 ε 均匀探索。**行为策略** β 决定数据从哪里来；**目标策略**在 Q-learning Target 中通过 max 表示。两者不同，所以 Q-learning 属于 **Off-Policy**。

例如旧值 $Q(s,a)=4$、奖励 $r=1$、下一状态最大 Q 为 6、$\gamma=0.9$、$\eta=0.2$，则

$$y = 1 + 0.9\times 6 = 6.4, \qquad Q_{\text{new}} = 4 + 0.2(6.4-4) = 4.48.$$

3.4 三种 Target 并排比较

方法	一步 Target（非终止）	下一动作来自哪里?	标准设置中的范式
SARSA	$r+\gamma Q(s',a')$	行为策略采样的 a'	On-Policy
Expected SARSA	$r+\gamma\mathbb{E}_{a'\sim\pi}Q(s',a')$	目标策略的分布期望	可 On/Off-Policy
Q-learning	$r+\gamma\max_{a'}Q(s',a')$	贪心最优性备份	Off-Policy

容易混淆的概念： Q-learning 的 Off-Policy 并不来自“使用了 Replay Buffer”；表格 Q-learning 即使每步立即更新，也可以是 Off-Policy。经验回放只是进一步提高数据复用、改变采样相关性的一种机制。

3.5 何时能保证收敛?

经典有限状态与动作表格 Q-learning 在固定 MDP、有界奖励、各状态动作对得到充分访问以及适当逐步衰减学习率（例如 Robbins–Monro 型条件）等假设下可收敛到 Q^*。**神经网络函数逼近、有限覆盖、非平稳环境、多步离策略自举等情形不直接受此定理保证**。 这一边界对随后理解 DQN 和 SAC 十分重要。Watkins & Dayan (1992)。

4. DQN：把表格 Q-learning 扩展到大状态空间

4.1 从表格到神经网络的 TD Regression

对于高维观测，无法为每个 s,a 存储表格值，用 $Q_\phi(s,a)$ 近似。单样本目标为

$$y = r + \gamma(1-d)\max_{a'} Q_{\bar\phi}(s',a'),$$

并最小化

$$\boxed{J_Q(\phi) = \frac{1}{2}\mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D}}\left[\left(Q_\phi(s,a) - \text{stopgrad}(y)\right)^2\right].}$$

这与后面 SAC 的 Critic Loss 在数学结构上非常相似：都通过 TD Target 对 Q 做回归。差别主要在于**下一动作如何选取、是否加入熵及是否显式学习 Actor**。

4.2 Replay Buffer 为什么有用?

顺序收集的数据彼此高度相关，若用刚产生的一条转移立即更新，神经网络容易受到短期采样分布影响。经验池

$$\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1}, d_t)\}_{t=1}^{N}$$

允许从过去交互随机抽取 Mini-Batch，减弱时间相关性并多次复用同一转移。**Replay Buffer 不能消除分布偏移：**若某些动作从未尝试，存储更多相似轨迹并不会自动提供这些动作的可靠 Q 标签。

4.3 Target Network 为什么有用?

若预测 $Q_\phi(s,a)$ 和目标 $r + \gamma \max_{a'} Q_\phi(s',a')$ 均随 ϕ 同步剧烈变化，学习目标会不断移动。DQN 使用相对慢变化的目标网络 $Q_{\bar{\phi}}$，既可以周期性复制 $\bar{\phi} \leftarrow \phi$，也可以采用软更新

$$\boxed{\bar{\phi} \leftarrow (1-\tau)\bar{\phi} + \tau\phi, \qquad 0 < \tau \ll 1.}$$

前者常见于经典 DQN；后者是现代连续控制 SAC/TD3 常见选择。目标网络降低目标漂移，但不能从根本上消除函数逼近偏差。

4.4 为什么最大化会放大 Q 高估?

设 $\widehat{Q}(s,a) = Q(s,a) + \epsilon_a$，哪怕 ϵ_a 对每个动作均值为零，**先取最大值再求期望**也可能偏高：

$$\mathbb{E}\left[\max_a (Q_a + \epsilon_a)\right] \geq \max_a \mathbb{E}[Q_a + \epsilon_a] = \max_a Q_a.$$

这里的 max 对随机变量是凸函数，故有 Jensen 不等式。动作越多、估计噪声越大，过高估计风险往往越突出。

4.5 Double Q-learning、Double DQN 与 SAC 的双 Q 不是同一公式

核心思想是把“动作选择”和“动作评价”拆开：

- **表格 Double Q-learning：**随机使用两套 Q 表之一选动作，再由另一套 Q 表评价，以降低同一估计器进行两项操作时的高估偏差。
- **Double DQN：** 在线网络选下一动作，目标网络评价该动作：

$$\boxed{a^* = \arg\max_{a'} Q_\phi(s',a'), \qquad y_{\text{DDQN}} = r + \gamma(1-d)Q_{\bar{\phi}}(s',a^*).}$$

- **SAC 的 Clipped Double Q：** 在给定 a' 后使用两个 Critic 的最小估计 $\min(Q_{\bar{\phi}_1}, Q_{\bar{\phi}_2})$。它以保守方式减轻高估，但可能引入低估。

这三者相关但**不能互换名称**；后面还会给出 SAC 公式。van Hasselt (2010)；van Hasselt et al. (2016)。

4.6 为什么深度价值学习仍可能不稳定?

在函数逼近、Bootstrapping 和 Off-Policy 数据同时出现时，可能形成常说的 **Deadly Triad**：有限采样导致的 Q 误差被下一步 Target 反复传播；策略又依据错误 Q 选择新动作，出现反馈放大。Replay、Target Network、双 Q 缓解部分机制，但不自动赋予全局收敛保证。

5. 连续动作为什么推动我们走向 Actor-Critic?

5.1 动作最大化成为内层优化问题

传统 Q-learning 要求计算

$$\max_{a' \in \mathcal{A}} Q_\phi(s', a').$$

对离散动作可以枚举；对机器人 $a \in \mathbb{R}^d$（例如机械臂与灵巧手的多个关节目标或力矩），高维动作很难枚举，而且 $Q_\phi(s,a)$ 往往非凸。每个 TD Target 都重新做一个高维内层优化，通常计算昂贵且可能陷入局部极值。

5.2 Actor 是一个可学习的动作优化器

引入参数化 Actor $a = \mu_\theta(s)$，用网络近似最优动作选择器；Critic 仍估计 $Q_\phi(s,a)$。对于确定性 Actor，典型目标为

$$J_\mu(\theta) = -\mathbb{E}_{s\sim\mathcal{D}}[Q_\phi(s, \mu_\theta(s))],$$

$$\nabla_\theta J_\mu = -\mathbb{E}_s \left[\nabla_a Q_\phi(s,a)|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s) \right].$$

这使得动作优化可以**摊销（amortize）**成一次前向传播，而非每次求解 $\arg\max_a Q$。DDPG 是确定性 Off-Policy Actor-Critic 的代表；TD3 在此基础上用双 Critic、延迟 Actor 更新及目标策略平滑等方法改善稳定性。

5.3 为什么 SAC 进一步使用随机 Actor?

确定性 Actor 缺少内生的策略熵；通常需要另外施加探索噪声。SAC 直接输出随机动作分布 $\pi_\theta(a \mid s)$，在最优化 Q 的同时优化熵：

$$\max_\pi \ \mathbb{E}_{a\sim\pi}[Q(s,a)] + \alpha \mathcal{H}(\pi(\cdot \mid s)).$$

注意：连续动作不“必然要求” Actor，某些离散化、采样优化或隐式策略方法依然可能适用；这里说明的是 SAC 所采用的高效、可微的工程路径。

6. 最大熵强化学习：从奖励最大化到奖励与熵联合优化

6.1 策略熵与温度

对离散动作，$\mathcal{H}(\pi) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$；对连续动作，微分熵为

$$\boxed{\mathcal{H}(\pi(\cdot \mid s)) = -\int_{\mathcal{A}} \pi(a \mid s) \log \pi(a \mid s)\, da = \mathbb{E}_{a\sim\pi}[-\log \pi(a \mid s)].}$$

连续微分熵允许为负数，并依赖动作坐标尺度；不可将其简单理解为“动作随机程度的无单位绝对值”。对高斯 $\mathcal{N}(\mu, \sigma^2)$：$\mathcal{H} = \frac{1}{2}\log(2\pi e \sigma^2)$。

引入温度 $\alpha \geq 0$，目标变为

$$\boxed{J_{\mathrm{ME}}(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t \big(r_t - \alpha \log \pi(a_t \mid s_t) \big) \right].}$$

α 较大时倾向维持更高熵；$\alpha \to 0^+$ 时趋近普通奖励目标。熵项依赖**当前策略**，不能误认为环境固定给出的奖励。

6.2 SAC 采用的 Soft Q 与 Soft V 约定

为了与现代 SAC Target 严格对应，本文约定

$$V^{\pi}_{\text{soft}}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^t(r_t - \alpha\log\pi(a_t \mid s_t)) \mid s_0 = s\right],$$
$$Q^{\pi}_{\text{soft}}(s,a) = \mathbb{E}_{\pi}\left[r_0 + \sum_{t=1}^{\infty}\gamma^t(r_t - \alpha\log\pi(a_t \mid s_t)) \mid s_0 = s, a_0 = a\right].$$

关键： Soft V 包含**当前动作**的熵；Soft Q 固定当前动作，熵从下一时刻起累积。因此

$$\boxed{V^{\pi}_{\text{soft}}(s) = \mathbb{E}_{a\sim\pi}\left[Q^{\pi}_{\text{soft}}(s,a) - \alpha\log\pi(a \mid s)\right],}$$

$$\boxed{Q^{\pi}_{\text{soft}}(s,a) = r(s,a) + \gamma\mathbb{E}_{s'}[V^{\pi}_{\text{soft}}(s')].}$$

合并得到 **Soft Bellman Expectation Equation**：

$$\boxed{Q^{\pi}_{\text{soft}}(s,a) = r(s,a) + \gamma\mathbb{E}_{s',a'\sim P,\pi}\left[Q^{\pi}_{\text{soft}}(s',a') - \alpha\log\pi(a' \mid s')\right].}$$

将此式与第 1 节的普通 Bellman Expectation Equation 对照，唯一结构性变化是下一状态价值中增加 $-\alpha\log\pi$。

7. Hard Bellman 与 Soft Bellman：从 Max 到 LogSumExp

7.1 有限离散动作中的最优 Soft Value

考虑**状态 s 固定**，假定动作有限、$\alpha > 0$，并给定候选 Q。对策略分布 $p_a \geq 0, \sum_a p_a = 1$，最大化

$$F(p) = \sum_a p_a Q(s,a) - \alpha\sum_a p_a\log p_a.$$

加入约束的拉格朗日乘子 λ，令

$$\frac{\partial}{\partial p_a}\left(F(p) + \lambda(\sum_a p_a - 1)\right) = Q(s,a) - \alpha(\log p_a + 1) + \lambda = 0.$$

得到 Boltzmann 分布

$$\boxed{p^*_a = \frac{\exp(Q(s,a)/\alpha)}{\sum_b \exp(Q(s,b)/\alpha)}.}$$

代回最优目标得到 **LogSumExp**：

$$\boxed{\max_p F(p) = \alpha\log\sum_a \exp\left(Q(s,a)/\alpha\right).}$$

因此，有限动作空间中的最优 Soft Bellman 算子是

$$\boxed{(\mathcal{T}^*_{\text{soft}}Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}\left[\alpha \log \sum_{a'} e^{Q(s',a')/\alpha}\right].}$$

此处的 Q 是上节约定的 Soft Q（当前动作不包括熵）；**不是**普通 Q-learning Q^* 无条件直接加一个 LogSumExp。

7.2 为什么 $\alpha \to 0$ 时恢复 Max?

令 $M = \max_i q_i$，n 为有限动作数：

$$\alpha \log \sum_i e^{q_i/\alpha} = M + \alpha \log \sum_i e^{(q_i - M)/\alpha}.$$

因为 $1 \le \sum_i e^{(q_i-M)/\alpha} \le n$，所以

$$\boxed{M \le \alpha \log \sum_i e^{q_i/\alpha} \le M + \alpha \log n,}$$

从而

$$\boxed{\lim_{\alpha \to 0^+} \alpha \log \sum_i e^{q_i/\alpha} = \max_i q_i.}$$

这给出直观桥梁：**Hard Max 是零温度极限，Soft Max 使用熵保留多个高价值候选动作**。

7.3 连续动作时的积分与边界条件

连续动作的形式是

$$V^*_{\text{soft}}(s) = \alpha \log \int_{\mathcal{A}} \exp(Q^*_{\text{soft}}(s,a)/\alpha)\, da.$$

它需满足积分有限，并且依赖所采用的动作基准测度；改变动作缩放会影响连续微分熵及归一化常数。对于高维非凸 Q，这个积分通常难以计算。**SAC 不会在训练中直接数值积分求 V^*_{soft}，而是学习一个 Actor 来近似相应的分布**。

7.4 Soft Optimality 与 Soft Policy Evaluation 必须分开

形式	解决的问题	下一步怎么处理动作?
$\mathcal{T}^\pi Q$	给定策略 π 的评估	$\mathbb{E}_{a' \sim \pi}$
$\mathcal{T}^* Q$	无熵最优控制	$\max_{a'}$
$\mathcal{T}^*_{\text{soft}} Q$	熵正则最优控制	$\alpha \log \sum e^{Q/\alpha}$（离散）
SAC Critic Target	对**当前 Actor**做近似 Soft Evaluation	$a' \sim \pi_\theta$，减 $\alpha \log \pi_\theta$

这张表是理解 Q-learning 与 SAC 关系的关键。SAC 的 Critic 更新**不是**把 Q-learning 的 max 直接替换成一个可显式计算的 LogSumExp，而是通过 Actor 来近似 Soft Policy Iteration。

8. Soft Policy Improvement：为什么出现 Boltzmann Policy 与 KL?

8.1 从最优分布到可训练的策略

固定某个 s 与旧策略的 Soft Q：$Q_{\text{old}}(s,a) = Q^{\pi_{\text{old}}}_{\text{soft}}(s,a)$。理想的策略改进是

$$\pi_{\text{new}} \in \arg\max_{\pi} \left\{ \mathbb{E}_{a\sim\pi}[Q_{\text{old}}(s,a)] + \alpha \mathcal{H}(\pi(\cdot \mid s)) \right\}.$$

在适当的可积性与密度假设下，连续动作中仍有

$$\boxed{\pi_B(a \mid s) = \frac{\exp(Q_{\text{old}}(s,a)/\alpha)}{Z_{\text{old}}(s)}, \quad Z_{\text{old}}(s) = \int_{\mathcal{A}} e^{Q_{\text{old}}(s,a')/\alpha}\, da'.}$$

然而 $Z_{\text{old}}(s)$ 对高维动作难以计算，且从 π_B 直接采样也不容易。

8.2 反向 KL 将不可计算的归一化常数消去

使用参数化 Actor π_θ，令其近似上述 Boltzmann Policy。展开 $D_{\text{KL}}(\pi_\theta \| \pi_B)$：

$$\begin{aligned} D_{\text{KL}}(\pi_\theta \| \pi_B) &= \mathbb{E}_{a\sim\pi_\theta}\left[\log \pi_\theta(a \mid s) - \log \pi_B(a \mid s)\right] \\ &= \mathbb{E}_{a\sim\pi_\theta}\left[\log \pi_\theta(a \mid s) - Q_{\text{old}}(s,a)/\alpha\right] + \log Z_{\text{old}}(s). \end{aligned}$$

因为 $\log Z_{\text{old}}$ 对 θ 是常数，最小化 KL 等价于

$$\boxed{\min_\theta \mathbb{E}_{a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_{\text{old}}(s,a)\right].}$$

这就是 SAC Actor Loss 的理论原型。这里采用 $D_{\text{KL}}(\pi_\theta \| \pi_B)$，并非 $D_{\text{KL}}(\pi_B \| \pi_\theta)$；KL 方向影响近似策略的行为。

8.3 精确 Soft Policy Iteration 的改进性质

假设 π_{old} 属于允许的策略族，并且每个状态的 KL 最小化精确实现，则新的 Soft Value 不小于旧策略在当前 Q 上的 Soft Value：

$$\mathbb{E}_{a\sim\pi_{\text{new}}}[Q^{\pi_{\text{old}}}_{\text{soft}}(s,a) - \alpha \log \pi_{\text{new}}(a \mid s)] \geq V^{\pi_{\text{old}}}_{\text{soft}}(s).$$

定义 Soft Policy Evaluation 算子

$$(\mathcal{T}^{\pi}_{\text{soft}} Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s',a'\sim P,\pi}[Q(s',a') - \alpha \log \pi(a' \mid s')].$$

它在适当有界性假设下具有单调性和 γ-压缩性，因此反复应用新策略的算子可得

$$\boxed{Q^{\pi_{\text{new}}}_{\text{soft}}(s,a) \geq Q^{\pi_{\text{old}}}_{\text{soft}}(s,a).}$$

这是**精确评估与精确改进**下的结论。实际 SAC 使用有限数据、神经网络及随机梯度，不具有由此自动推出的全局单调性或收敛保证。

9. 从 Soft Policy Iteration 到现代 SAC

9.1 两个循环对应两个神经网络

到此我们得到一个完整的逻辑链：

1. **Soft Policy Evaluation：** 通过 Soft Bellman Equation 学习当前策略的 Q^{π}_{soft}。
2. **Soft Policy Improvement：** 借助 Q 更新 Actor，逼近 Boltzmann 分布。

采用神经网络近似

$$\boxed{\text{Actor: } \pi_\theta(a \mid s), \qquad \text{Critics: } Q_{\phi_1}(s,a),\ Q_{\phi_2}(s,a).}$$

Critic 接受状态动作对、输出标量；Actor 接受状态、输出条件动作分布。奖励先用于训练 Q，再通过 Q 的动作梯度间接指导 Actor；一般不会把环境模拟器中的奖励函数直接端到端反向传播到 Actor。

9.2 为什么 SAC 可以复用历史数据?

假设 MDP 固定，则 $P(s' \mid s,a)$ 不取决于当时由哪个行为策略 β 采样到了 (s,a)。因此经验池中由旧行为策略生成的 (s,a,r,s',d) 仍然可用于估计一步环境条件期望；而目标中**下一动作**需要使用**当前策略**：

$$y = r + \gamma(1-d)\left[Q(s',a') - \alpha \log \pi_\theta(a' \mid s')\right], \qquad a' \sim \pi_\theta(\cdot \mid s').$$

元素	来自哪里?	是否需要当前 Actor 重新计算?
(s,a,r,s',d)	Replay Buffer 历史转移	不需要
a'	当前 Actor 在 s' 采样	需要
$\log \pi_\theta(a' \mid s')$	当前 Actor 的对数密度	需要
$Q(s',a')$	目标 Critic	需要

Replay Buffer 改善数据效率，但不会使 Bellman 更新在任何历史分布下都无偏。即使条件环境转移正确，**状态动作覆盖不足、Critic 外推、函数逼近与自举误差**仍然可能损坏更新；Actor 的状态采样通常也使用 Buffer 分布而非精确的当前折扣占用分布。

9.3 与 PPO 的 On-Policy 属性区分

PPO 使用旧策略产生的动作及概率比 $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$。标准 PPO 通常重复使用少量近期 Rollout Epoch，不能无限制地把任意历史动作当成当前策略数据。SAC 的目标是基于 Bellman 的 Off-Policy 策略评估和改进；它不需要 PPO 的概率比 Clipping。这不意味着 PPO 从数学上完全不可能利用旧数据，而是指其标准算法设定与重用范围不同。

10. 完整 SAC Critic：Target Network、双 Q 与 TD 回归

10.1 从 Soft Bellman 到单样本 TD Target

采用两个 Critic Q_{ϕ_i} 及慢更新 Target Critics $Q_{\bar{\phi}_i}$。对 Replay Buffer 中的样本 (s,a,r,s',d)，由 Actor 在下一状态采样 $a' \sim \pi_\theta(\cdot \mid s')$，构造

$$\boxed{y = r + \gamma(1-d)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_\theta(a' \mid s')\right].}$$

Critic 分别最小化

$$\boxed{J_{Q_i}(\phi_i) = \frac{1}{2}\mathbb{E}_{\mathcal{D}}\left[(Q_{\phi_i}(s,a) - \text{stopgrad}(y))^2\right], \quad i = 1, 2.}$$

y 是随机单样本近似，损失是平方 TD Error 的回归。Target 分支必须停止梯度；否则训练一个 Q 的同时又让 Target 通过相同反向图更新，将改变这里的半梯度（semi-gradient）设定。

10.2 双 Q 的取最小值解决什么?

如果 Actor 总是倾向 Q 高估的动作，这种偏差可能进一步通过 Bootstrap 回传。SAC 在评估给定下一动作 a' 时采用 $\min_j Q_{\bar{\phi}_j}(s', a')$，减少因正误差带来的过度乐观。但要记住：**取最小不等于“无偏”**，两个 Critic 误差相关时仍会存在风险，而且可能系统性低估价值。

经典 Q-learning 的 $\max_{a'}$ 是**最优性备份**；SAC 取 $\min_j$ 是**估计器融合**。一个在动作维度求最大，另一个在 Q 网络维度取最小，两者绝对不能混淆。

10.3 时间截断与真正终止

若 $d = 1$ 是 MDP 真实终止，则 $y = r$；若环境仅由于 Time Limit 达到而暂时截断（`truncated=True`），在无限时域或可继续的任务中通常仍需 Bootstrapping。实践中应区分 `terminated` 与 `truncated`，否则会错误低估末尾状态价值。有限时域任务若把剩余时间作为状态的一部分，需根据终止定义一致处理。

10.4 目标网络的软更新

训练每步或按设定间隔更新

$$\boxed{\bar{\phi}_j \leftarrow (1-\tau)\bar{\phi}_j + \tau\phi_j, \qquad 0 < \tau \ll 1.}$$

典型设置之一为 $\tau = 0.005$，但该值不具有普适最优性。软更新减少 Q Target 高频变化，不能替代合理的奖励尺度、探索覆盖与数值稳定处理。

11. 完整 SAC Actor：重参数化、动作边界与概率修正

11.1 从 KL 投影到 Actor Loss

用 Buffer 状态分布进行近似策略改进，常见 Actor Objective 为

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s\sim\mathcal{D} \\ a\sim\pi_\theta(\cdot|s)}}\left[\alpha \log \pi_\theta(a \mid s) - \min_{j=1,2} Q_{\phi_j}(s,a)\right].}$$

第一项鼓励适当熵，第二项鼓励高 Q 动作。训练 Actor 时通常固定 Critic **参数**，但必须保留 $\nabla_a Q_\phi(s,a)$，否则 Q 无法通过动作向 Actor 提供改进方向。也有实现用 Q_{ϕ_1} 代替最小值更新 Actor，需与具体实现核对。

11.2 Reparameterization Trick

若直接写 $a \sim \pi_\theta(a \mid s)$，随机采样依赖 θ，难以直接进行简单的逐样本路径微分。令 $\epsilon \sim \mathcal{N}(0, I)$ 与参数无关，网络输出 $\mu_\theta(s), \sigma_\theta(s)$：

$$\boxed{u_\theta(s,\epsilon) = \mu_\theta(s) + \sigma_\theta(s) \odot \epsilon,}$$

$$\boxed{a_\theta(s,\epsilon)=\tanh u_\theta(s,\epsilon).}$$

此时 $a_\theta=f_\theta(s,\epsilon)$ 对 θ 可微，ϵ 的采样分布固定。Actor Objective 写为

$$J_\pi(\theta)=\mathbb{E}_{s\sim\mathcal{D},\epsilon\sim\mathcal{N}(0,I)}\left[\alpha\log\pi_\theta(f_\theta(s,\epsilon)\mid s)-Q_\phi(s,f_\theta(s,\epsilon))\right].$$

为突出机制，此式的 Q_ϕ 可代表实际采用的一个 Critic 或 $\min_j Q_{\phi_j}$。其梯度含

$$-\underbrace{\nabla_a Q_\phi(s,a_\theta)}_{\text{动作方向上的价值梯度}}\underbrace{\frac{\partial a_\theta}{\partial\theta}}_{\text{Actor 如何改变动作}},$$

并同时包含 $\alpha\log\pi_\theta(a_\theta\mid s)$ 的**全导数**，即显式策略参数依赖与动作路径依赖两部分。SAC 的路径梯度与 PPO 常用的 Score-Function Gradient 不同，二者不应简化为“随机策略对确定性策略”。

11.3 Tanh Gaussian 的 Log Probability 必须修正

对于 $u\sim\mathcal{N}(\mu_\theta(s),\operatorname{diag}\sigma_\theta^2(s))$，$a=\tanh u$，变量变换给出

$$\pi_\theta(a\mid s)=p_U(u\mid s)\left|\det\frac{\partial a}{\partial u}\right|^{-1}.$$

由于 $\partial a_i/\partial u_i=1-\tanh^2 u_i$，

$$\boxed{\log\pi_\theta(a\mid s)=\log\mathcal{N}(u;\mu_\theta(s),\Sigma_\theta(s))-\sum_{i=1}^{d}\log(1-\tanh^2 u_i).}$$

不能用 $\log\mathcal{N}(a;\mu,\Sigma)$ 代替。数值上可使用恒等式

$$\log(1-\tanh^2 u)=2\big(\log 2-u-\operatorname{softplus}(-2u)\big)$$

避免 $\tanh u$ 接近 ±1 时的数值不稳定。

11.4 从归一化动作到机器人实际动作

若第 i 维执行动作 $a_i^{\text{env}}=c_i+b_i\tanh u_i$，其中

$$c_i=\frac{a_{\max,i}+a_{\min,i}}{2},\qquad b_i=\frac{a_{\max,i}-a_{\min,i}}{2}>0,$$

则相对于**实际动作变量**的密度还需要减去 $\sum_i\log b_i$：

$$\log\pi_\theta(a^{\text{env}}\mid s)=\log\mathcal{N}(u;\mu,\Sigma)-\sum_i\log\left[b_i(1-\tanh^2 u_i)\right].$$

如果算法始终在归一化 $[-1,1]^d$ 空间定义策略密度、熵目标与 Q 的动作输入，再由环境封装层进行固定缩放，就应一以贯之地使用这一约定。两个约定仅差常数项，但会影响熵值和温度解释，尤其是自动温度学习。

12. Automatic Entropy Tuning：为什么学习温度 α？

12.1 目标熵约束与拉格朗日乘子

固定 α 时，奖励尺度和动作维度改变可能显著影响探索。现代 SAC 可以约束平均策略熵

$$\max_{\pi} J_R(\pi) \quad \text{s.t.} \quad \bar{\mathcal{H}}(\pi) \geq \mathcal{H}_{\text{target}}.$$

对应的 Lagrangian（概括性平均约束形式）为

$$\mathcal{L}(\pi, \alpha) = J_R(\pi) + \alpha(\bar{\mathcal{H}}(\pi) - \mathcal{H}_{\text{target}}), \qquad \alpha \geq 0.$$

优化结构为 $\max_\pi \min_{\alpha \geq 0} \mathcal{L}(\pi, \alpha)$。原 SAC 温度推导可采用逐时刻约束；这里用平均熵表示其训练动机，不能把不同形式在所有情形下视为严格等价。

12.2 温度损失与方向检查

固定 Actor 时，最小化对偶相关目标

$$\boxed{J_\alpha(\alpha) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\theta}\left[\alpha\big(-\log \pi_\theta(a \mid s) - \mathcal{H}_{\text{target}}\big)\right].}$$

梯度为 $\partial J_\alpha / \partial \alpha = \bar{\mathcal{H}}(\pi) - \mathcal{H}_{\text{target}}$。若当前熵太低，梯度为负，最小化使 α 增加；若熵太高，则 α 减少。**负的连续目标熵是允许的**，不应套用离散动作熵非负的直觉。

12.3 为什么优化 $\eta = \log \alpha$?

用 $\alpha = e^\eta > 0$ 保证正数；严格重参数化目标为

$$J_\eta(\eta) = -\mathbb{E}\left[e^\eta(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})\right].$$

不少实现使用梯度尺度不同但内部零点一致的代理目标

$$\boxed{J_\eta^{\text{sur}}(\eta) = -\mathbb{E}\left[\eta(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})\right].}$$

更新温度时必须将 $\log \pi_\theta$ 视为常数（停止梯度），否则温度损失可能意外更新 Actor。常见启发式 $\mathcal{H}_{\text{target}} = -d$（对 d 维归一化动作）**不是定理**；高维灵巧手、动作相关维度和动作约束场景应审慎解释。

13. 现代 SAC 的完整算法流程

13.1 主要模块和三个损失

模块	记号	功能
Stochastic Actor	$\pi_\theta(a \mid s)$	生成高价值且保留适度熵的连续动作
Twin Critics	Q_{ϕ_1}, Q_{ϕ_2}	估计 Soft Q，缓解高估
Target Critics	$Q_{\bar{\phi}_1}, Q_{\bar{\phi}_2}$	提供变化较慢的 TD Target
Replay Buffer	$\mathcal{D}$	保存历史转移，Off-Policy 重用
Temperature	$\alpha = e^\eta$	以目标熵调节探索权重

三个损失分别为 $J_{Q_1} + J_{Q_2}$、J_π、J_η。其关系不是“一个网络同时做所有事”，而是**当前策略评估 → Actor 改进 → 温度调节 → Target 缓慢追踪**。

13.2 一次更新究竟发生了什么?

1. 使用当前策略（初始阶段也可随机探索）与环境交互，存储 $(s, a, r, s', \text{terminated}, \text{truncated})$。
2. 从 $\mathcal{D}$ 抽取一个 Mini-Batch；从当前 Actor 在各 s' 上重采样 a'，使用 Target Critics 和 α 计算 TD Target y，停止 Target 分支梯度。
3. 使用 y 分别回归两个 Critic 的 $Q_{\phi_i}(s, a)$。
4. 在同一批状态 s 上从当前 Actor 重采样 $\tilde{a}$，以 $\alpha \log \pi_\theta(\tilde{a} \mid s) - \min_i Q_{\phi_i}(s, \tilde{a})$ 更新 Actor；Critic 参数不更新但保留动作梯度。
5. 用停止梯度的对数概率更新 $\log \alpha$，调整探索强度。
6. 通过 Polyak Averaging 软更新 Target Critics。然后继续交互与优化；更新次数与交互次数比例可以不同。

下面是**数学伪代码**（非可执行 Python，也不构成实验章节）:

```
Initialize actor pi_theta; critics Q_phi1,Q_phi2
Initialize target critics Q_barphi1,Q_barphi2; replay D; eta=log(alpha)
Repeat:
    Collect environment transition and append to D
    For each gradient update:
        Sample (s,a,r,s_next,terminated,truncated) from D
        a_next ~ pi_theta(. | s_next)
        y = r + gamma*(1-terminated)*(
                min_i Q_barphi_i(s_next,a_next) - alpha*log pi_theta(a_next|s_next))
        Update each Q_phi_i by gradient of (Q_phi_i(s,a)-stopgrad(y))^2
        a_pi ~ pi_theta(. | s)  # reparameterized sample
        Update actor by gradient of alpha*log pi_theta(a_pi|s)-min_i Q_phi_i(s,a_pi)
        Update eta by entropy dual loss (detach log probability)
        barphi_i <- (1-tau)*barphi_i + tau*phi_i
```

这里 `terminated` 与 `truncated` 的区别、动作空间缩放、Tanh Jacobian、Actor 更新时的 Critic 参数冻结及 Replay Buffer 覆盖度，都是算法定义不可忽略的细节。

13.3 SAC v1 与现代 SAC：是否显式学习 V?

最初版本还训练状态价值网络 $V_\psi(s)$:

$$J_V(\psi) = \frac{1}{2}\mathbb{E}_s\left[\left(V_\psi(s) - \text{stopgrad}(\min_i Q_{\phi_i}(s, \tilde{a}) - \alpha \log \pi_\theta(\tilde{a} \mid s))\right)^2\right],$$

Q Target 为 $y_Q = r + \gamma(1-d)V_{\bar{\psi}}(s')$。现代常用版本移除显式 V 网络，直接使用 Target Critics 与下一动作的对数概率构造 y；因此常见实现只有 Actor、两个 Critic、两个目标 Critic 和温度参数。不要把“没有 V 网络”误读成“不再使用 Soft V 的数学概念”。

14. 算法之间到底有什么联系?

方法	典型价值更新	Actor / 动作选择	数据范式	主要设计点
SARSA	下一行为动作的 $Q(s', a')$	由行为策略选动作	On-Policy（标准）	学习执行中策略的价值
Q-learning	$\max_{a'} Q(s', a')$	贪心 + 探索	Off-Policy	Bellman Optimality
DQN	神经网络 + $\max_{a'} Q_{\bar{\phi}}$	离散 $\arg\max$	Off-Policy	Replay 与目标网络

方法	典型价值更新	Actor / 动作选择	数据范式	主要设计点
Double DQN	在线网络选、目标网络评	离散 $\arg\max$	Off-Policy	缓解 Max 高估
DDPG	确定性 Actor 的 Target Q	确定性 Actor + 探索噪声	Off-Policy	连续动作策略梯度
TD3	双 Q、保守 Target	延迟确定性 Actor	Off-Policy	双 Q 与目标平滑
SAC	Soft Bellman + 双 Q	随机 Actor + 熵	Off-Policy	Soft Policy Iteration
PPO	通常使用 V 与 Advantage	随机策略概率比更新	On-Policy（标准）	Clipped Policy Objective

14.1 Q-learning 与 SAC 的最关键区别

经典 Q-learning 尝试直接逼近最优 Q，并在 Target 中计算 $\max_{a'} Q(s', a')$；SAC 的 Critic 估计**当前随机 Actor 的 Soft Q**，Actor 则利用 Q 做熵正则策略改进：

$$\boxed{\begin{aligned}&\text{Q-learning: } y = r + \gamma \max_{a'} Q(s', a'),\\ &\text{SAC: } y = r + \gamma \left[\min_j Q_{\bar{\phi}_j}(s', a') - \alpha \log \pi(a' \mid s')\right], \quad a' \sim \pi.\end{aligned}}$$

为了简洁，式中省略真实终止因子；实际应乘 $1 - d$。SAC 中的 $\min_j$ 不是对动作取最小，它只在多个价值估计器中选择较保守的一个。

14.2 PPO 与 SAC：Score Function 和 Pathwise Gradient

PPO 的常用策略更新方向由 $\nabla_\theta \log \pi_\theta(a \mid s)\, \hat{A}(s, a)$ 构造，辅以策略概率比 Clipping；SAC 通过重参数化动作让 $\nabla_a Q(s, a)\, \partial a / \partial \theta$ 传到 Actor，同时加上熵梯度。前者通常不要求 Critic 对动作可微；后者标准连续动作实现通常要求。PPO 的 On-Policy 大规模并行采样在仿真里可能有墙钟时间优势；SAC 在真实交互昂贵时往往有较高样本效率。**样本效率不等于墙钟训练效率**。

15. 机器人控制中的适用边界与常见误解

15.1 为什么机器人任务特别需要重视 Q 外推？

机械臂、灵巧手操作往往有接触模式切换、强非线性、稀疏奖励、视觉状态误差与动作边界。Replay Buffer 缺少某些接触状态或恢复动作时，Critic 可以在未覆盖区域给出很高的错误值；Actor 梯度会主动寻找这些错误高值区域，从而形成 **Actor–Critic Feedback Loop**。双 Q 只缓解部分高估，不能替代状态动作覆盖、奖励检查、安全约束与在线分布监测。

15.2 关节位置、速度、力矩与归一化动作

策略输出 $[-1, 1]^d$ 通常并非直接执行的关节位置或力矩；还需要根据控制接口进行缩放、偏置和物理限幅。动作饱和可能让 Tanh 导数变小，动作尺度影响高斯密度与目标熵。将动作表示、控制频率、观测延迟、执行器限幅清晰写入 MDP，远比只看 J_π 的数学形式更重要。

15.3 什么情况下不宜直接套用标准 SAC？

标准 SAC 假设可从 Actor 分布采样并计算其 Log Probability，还通常依赖动作路径可微性。若使用离散动作、复杂的隐式生成策略、Flow Matching 动作头、部分可观测长历史策略或强安全约束，则需要明确目标与策略

参数化是否仍然成立；不能把 $\alpha \log \pi$ 直接加到无法计算密度的策略上并称其为标准 SAC。历史数据并不能自动解决 Offline RL 的分布外动作价值外推问题，Offline 与 Online Off-Policy RL 也不是同义词。

15.4 五个常见误解

1. **“Q-learning 只有用了 Replay Buffer 才是 Off-Policy。”** 错；Off-Policy 来自行为与学习目标的区别。
2. **“Soft Q-learning 就是直接把 Q-learning 的 Max 换成 Actor 采样。”** 不严谨；需区分最优 Soft Bellman 与给定策略的 Soft Evaluation。
3. **“Double Q、Double DQN 和 SAC 的 Min-Q 完全相同。”** 错；选择与评价的分离方式不同。
4. **“α 大一定提高回报。”** 错；较大熵系数可能牺牲任务回报，其合适数值与奖励和动作尺度有关。
5. **“Off-Policy 经验复用保证稳定和安全。”** 错；覆盖不足、Bootstrapping、非平稳性以及实际执行约束仍可能带来问题。

16. 全部推导的最短闭环

下面六个公式足以概括从经典 Q-learning 到现代 SAC 的核心变化。

(1) 经典 Bellman Optimality：

$$\boxed{Q^*(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}[\max_{a'} Q^*(s',a')].}$$

(2) Q-learning 单样本 TD 更新：

$$\boxed{Q(s,a) \leftarrow Q(s,a) + \eta \left[r + \gamma(1-d) \max_{a'} Q(s',a') - Q(s,a) \right].}$$

(3) 最优 Soft Value（有限离散动作）：

$$\boxed{V^*_{\text{soft}}(s) = \alpha \log \sum_a \exp(Q^*_{\text{soft}}(s,a)/\alpha),}$$

其零温度极限恢复 $\max_a Q(s,a)$。

(4) 给定策略的 Soft Bellman Evaluation：

$$\boxed{Q^\pi_{\text{soft}}(s,a) = r + \gamma \mathbb{E}_{s',a' \sim P,\pi}[Q^\pi_{\text{soft}}(s',a') - \alpha \log \pi(a' \mid s')].}$$

(5) SAC 的双 Critic 与重参数化 Actor：

$$\boxed{\begin{aligned} y &= r + \gamma(1-d) \left[\min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_\theta(a' \mid s') \right], \\ J_\pi &= \mathbb{E}_{s,\epsilon} \left[\alpha \log \pi_\theta(f_\theta(s,\epsilon) \mid s) - \min_j Q_{\phi_j}(s, f_\theta(s,\epsilon)) \right]. \end{aligned}}$$

(6) 目标熵与温度调节：

$$\boxed{J^{\text{sur}}_\eta = -\mathbb{E}\left[\eta(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})\right], \quad \eta = \log \alpha.}$$

一句话总结： Q-learning 通过 **Hard Bellman Optimality + TD Bootstrapping** 学最优动作价值；SAC 通过 **Soft Bellman Policy Evaluation + Actor Policy Improvement + Entropy Tuning** 在连续动作空间学习随机控制策略。共享的是 Bellman 与 TD，关键改变是如何优化下一动作以及如何保持策略熵。

参考资料与延伸阅读

1. Watkins, C. J. C. H., & Dayan, P. (1992). **Q-learning.** *Machine Learning*, 8, 279–292. DOI。表格 Q-learning 与经典收敛定理。
2. Sutton, R. S., & Barto, A. G. (2018). **Reinforcement Learning: An Introduction**, 2nd ed. 公开教材。Bellman 方程、TD、SARSA、Q-learning 与策略迭代。
3. Mnih, V., et al. (2015). **Human-level control through deep reinforcement learning.** *Nature*. DOI。DQN、经验回放、目标网络。
4. van Hasselt, H. (2010). **Double Q-learning.** *NeurIPS*. 论文。动作价值高估与双估计器。
5. van Hasselt, H., Guez, A., & Silver, D. (2016). **Deep Reinforcement Learning with Double Q-learning.** arXiv:1509.06461。Double DQN。
6. Lillicrap, T. P., et al. (2015). **Continuous control with deep reinforcement learning.** arXiv:1509.02971。确定性连续控制 Actor-Critic。
7. Fujimoto, S., van Hoof, H., & Meger, D. (2018). **Addressing Function Approximation Error in Actor-Critic Methods.** arXiv:1802.09477。TD3 与双 Q 控制误差。
8. Haarnoja, T., Tang, H., Abbeel, P., & Levine, S. (2017). **Reinforcement Learning with Deep Energy-Based Policies.** *ICML*. 论文。Soft Q-learning 与能量模型策略。
9. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). **Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.** arXiv:1801.01290。Soft Policy Iteration 与 KL 推导。
10. Haarnoja, T., et al. (2018). **Soft Actor-Critic Algorithms and Applications.** arXiv:1812.05905。自动温度与现代 SAC。
11. OpenAI Spinning Up. **Soft Actor-Critic.** 算法说明。常用实现约定与公式。

许可说明：原创课程内容按 CC BY 4.0 标注；对第三方文献的引用不改变其原有许可。本文为课程讲义，不含实验、数值结果或训练效果保证。Markdown 中公式采用 GitHub / Pandoc 常见的 LaTeX 数学格式；随附 PDF 与本 Markdown 使用同一版正文生成。